

Table of Contents

Week 2 — Classes & Objects

The Two Core Building Blocks of OOP

Lesson Overview

Hour Plan

Recap of Week 1 (5 min)

Part 1 — What Is a Class? (15 min)

Open with a question (2 min)

Definition

Five Real-World Class Examples

The Blueprint Analogy — Core

What a Class Does NOT Do

Part 2 — What Is an Object? (15 min)

Follow on from the smartphone question (2 min)

Definition

Class Template → Object with Real Data

Objects Are Independent — The Core OOP Insight

Three Independence Scenarios — Ask Students to Predict

Object State

Part 3 — Class vs Object: The Critical Difference (12 min)

Full Comparison Table

The Quick Test

Three Questions to Always Ask

Part 4 — Java as Illustration (10 min)

Reading a Class as a Concept Description

Creating Objects — Filling in the Template

How to Read One Line of Code as Three Concepts

The University Model for This Course

Test Checkpoint

Common Misconceptions

Extension Challenges

● Basic — Class or Object?

● Intermediate — Design a Class

Advanced — One Class, Many Contexts

Lesson Summary

Homework (No Java required)

Week 2 — Classes & Objects

The Two Core Building Blocks of OOP

Date: Wednesday, 10 June 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Classes & Objects — the fundamental unit of every OOP system
Date & Time	Wednesday, 10 June 2026 · 6:30 PM – 8:30 PM
Duration	2 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + guided illustration
Prerequisites	Week 1 — What is OOP, the 4 pillars, real-world mapping
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain Class and Object in plain words, give real-world examples from their own life, identify the class/object in any scenario, and understand why objects are independent of each other

Hour Plan

00:00 – 00:05	Recap of Week 1 + Homework check
00:05 – 00:20	Part 1 – What is a Class? (15 min)
00:20 – 00:35	Part 2 – What is an Object? (15 min)
00:35 – 00:47	Part 3 – Class vs Object: the critical difference (12 min)
00:47 – 00:57	Part 4 – Java as illustration: seeing the concept in code (10 min)
00:57 – 01:00	Checkpoint + Homework

Recap of Week 1 (5 min)

Ask students verbally — no pressure, just discussion:

1. "OOP organizes code as objects. What does an object bundle together?"
→ *data (attributes) + behavior (methods)*
 2. "What is the blueprint analogy we used for Class?"
→ *accept any blueprint example — house, recipe, cookie cutter*
 3. "Quick fire — the 4 pillars of OOP. Who can name them all?"
→ *Encapsulation · Inheritance · Polymorphism · Abstraction*
 4. "From homework — when you changed one Animal's sleeping status, did other animals change?"
→ *No. Each object is independent. This is exactly what we study today.*
-

Part 1 — What Is a Class? (15 min)

Open with a question (2 min)

*"Think of a smartphone — any smartphone.
Now imagine 30 different phones owned by 30 different people in this room.
What do ALL of those phones have in common?"*

Definition

A **class** is a description — a template — that defines:

- What **data** (attributes) every object of this type will carry
- What **actions** (methods) every object of this type can perform

A class is **not a real thing**. It is a **category description**.
It holds no actual data. It exists only to define structure.

Five Real-World Class Examples

Work through each one interactively. Ask students: "What attributes? What actions?" before revealing.

CLASS: Smartphone

Attributes: brand, model, storageGB, batteryPercent, isOn
Actions: call(), sendMessage(), takePhoto(), charge()

This is not YOUR phone. Not anyone's phone.
It is the description of what every smartphone has.

CLASS: Student

Attributes: name, age, studentId, major, gpa
Actions: enroll(), submitAssignment(), checkGrade()

Not Sokha. Not Dara. Not anyone specific.
Just the definition: what every student carries and can do.

CLASS: DrinkOrder (café)

Attributes: drinkType, size, sugar, ice, price
Actions: prepare(), deliver(), calculateTotal()

Not order #47. Not "iced Americano for Sokha."
Just the description of what every order looks like.

CLASS: BankAccount

Attributes: owner, accountNumber, balance, accountType
Actions: deposit(), withdraw(), checkBalance()

Not YOUR account. Not account #KH-4291.
Just the description of what every account has.

CLASS: Book (library)

Attributes: title, author, isbn, pages, isAvailable
Actions: checkOut(), returnBook(), reserve()

Not "Clean Code" by Robert Martin.
Not the specific copy on shelf B-12.
Just the description of what every book in the system has.

The Blueprint Analogy — Core

This is the most important analogy of the entire course. Take time here.

ARCHITECTURAL BLUEPRINT

An architect draws ONE blueprint for a house:

- 3 bedrooms, 2 bathrooms, kitchen, living room
- Dimensions: 12m × 8m
- Materials: concrete walls, tiled roof

The blueprint is NOT a house.

You cannot sleep in a blueprint.

You cannot cook in a blueprint.

The blueprint exists ONLY as a description.

From that ONE blueprint, builders construct:

- House #1 → yellow, owned by Sokha, has a garden
- House #2 → blue, owned by Dara, has a dog
- House #3 → white, owned by Chan, under renovation

Each house is REAL and DIFFERENT.

Painting house #1 yellow does NOT paint house #2.

Demolishing house #1 does NOT destroy the blueprint.

CLASS = the blueprint (structure only, no data)

OBJECT = each real house (actual data, actual state)

What a Class Does NOT Do

Be explicit — this clears up persistent confusion.

A class does NOT:

- X Store Sokha's name or Dara's age
- X Hold memory for any specific person's data
- X Run, execute, or "do" anything on its own
- X Represent any specific, real person or thing

A class DOES:

- ✓ Define: "every object of this type will have these attributes"
- ✓ Define: "every object of this type can perform these actions"
- ✓ Serve as the template used to build real objects
- ✓ Be the "type" label used in code (e.g., Student sokha)

Ask students: "If I say `Student.name` — am I getting a real name? Why not?"

Expected: No — Student is the class/category. It holds no actual name.

Part 2 — What Is an Object? (15 min)

Follow on from the smartphone question (2 min)

"Same 30 phones in the room. Now instead of 'what do they all share' — tell me about YOUR specific phone right now.

What brand? What battery level? Is it on? How much storage?"

Every student gives a different answer. That is the point.

"Each specific phone — with its own brand, battery, photos, contacts — is an **object**. The class described the category.

The object IS the real, specific thing."

Definition

An **object** is a real, specific instance built from a class.

It holds **actual data values** in all the attributes the class defined.

It can perform all the **actions** the class defined.

Each object has its **own independent copy** of every attribute.

Class Template → Object with Real Data

CLASS (template, no data)	OBJECT (real instance, actual data)
Smartphone	Sokha's phone
brand: ??? →	brand: "Samsung"
model: ??? →	model: "Galaxy A55"
storageGB: ??? →	storageGB: 128
batteryPercent: ??? →	batteryPercent: 73
isOn: ??? →	isOn: true

Smartphone			Dara's phone		
brand:	???	→	brand:	"Apple"	
model:	???	→	model:	"iPhone 15 Pro"	
storageGB:	???	→	storageGB:	256	
batteryPercent:	???	→	batteryPercent:	45	
isOn:	???	→	isOn:	true	

Same class. Two completely different objects with different data.

Objects Are Independent — The Core OOP Insight

This is the most important idea in this lesson. Spend real time here.

SCENARIO: Sokha and Dara both have a Smartphone object.

- Dara's battery dies. She turns her phone off.
 - Dara's `isOn` = `false`
 - Sokha's `isOn` = `???` → still `TRUE` (independent)
- Sokha takes 20 photos.
 - Sokha's `photoCount` increases
 - Dara's `photoCount` = `???` → unchanged (independent)
- Sokha drops her phone. The screen cracks. (object "damaged")
 - Sokha's phone has new state: `screenCracked` = `true`
 - Dara's phone = `???` → still fine (independent)
 - The Smartphone CLASS = `???` → completely unaffected (blueprint survives)

THE RULE:

Every object owns its own copy of every attribute.
 What happens to one object has NO effect on any other object.
 The class is never affected by changes to any of its objects.

Three Independence Scenarios — Ask Students to Predict

For each scenario, ask students: "What changes? What stays the same?"

SCENARIO A – Library

Class: Book

Object 1: "Clean Code" `isAvailable` = `true`

→ Sokha borrows Object 1 (checks it out)
Object 1: `isAvailable = false` ✓
Object 2: `isAvailable = ???` (students answer) → still `true` ✓

SCENARIO B – Bank Accounts

Class: `BankAccount`

Object 1: Sokha's account `balance = $500`

Object 2: Dara's account `balance = $200`

→ Sokha deposits \$300
Object 1: `balance = $800` ✓
Object 2: `balance = ???` (students answer) → still `$200` ✓

SCENARIO C – Café Orders

Class: `DrinkOrder`

Object 1: Order #47 – Large iced coffee, no sugar

Object 2: Order #48 – Hot tea, 2 sugars

→ Barista adds extra ice to Order #47
Object 1: `ice = "extra"` ✓
Object 2: `ice = ???` (students answer) → still `normal` ✓

Object State

An object's **state** is the current value of all its attributes at one specific moment.
State is expected to change — that is how objects “do things.”

Sokha's `BankAccount` – state over time:

Account created: `owner="Sokha", balance=500, transactions=0`

After deposit 200: `owner="Sokha", balance=700, transactions=1`

After withdraw 50: `owner="Sokha", balance=650, transactions=2`

The object's identity (it is still Sokha's account) never changes.
The object's state (balance, transaction count) evolves through actions.

Part 3 — Class vs Object: The Critical Difference (12 min)

Full Comparison Table

Walk through row by row. Ask students to give their own example for each row.

	CLASS	OBJECT
What is it?	A template / description	A real, specific thing
Exists as ...	A definition	A real instance in memory
Holds data?	No – only defines structure	Yes – holds actual values
How many?	Usually ONE per concept	Can be thousands
Example	"Smartphone" (the concept)	Sokha's Galaxy A55
	"Student" (the concept)	The real person Sokha
	"BankAccount" (the concept)	Account #KH-0042, \$650
	"DrinkOrder" (the concept)	Order #47, iced Americano
Created by?	A programmer (in code)	The program, when it runs
Real-world	Cookie cutter	Each individual cookie
	House blueprint	Each built house
	Recipe	Each meal you cook from it
	Passport template	Your specific passport

The Quick Test

"Can I hold it, touch it, or give it a specific value RIGHT NOW?"

- YES → it is an **Object** (real, specific, has data)
- NO → it is a **Class** (template, concept, no data)

PRACTICE – CLASS or OBJECT?

"Student"	→ Class (just a description)
"Sokha, age 20, GPA 3.8, major CS"	→ Object (real data, specific)
"Car"	→ Class
"My red 2022 Toyota Camry"	→ Object
"BankAccount"	→ Class
"Account #KH-4291 with \$1,200"	→ Object
"DrinkOrder"	→ Class
"Large iced Americano, no sugar, #47"	→ Object
"Book"	→ Class

Three Questions to Always Ask

Teach students to ask these questions whenever they encounter a class or object:

- Q1: "What type is this?" → gives you the Class name
- Q2: "What are the specific values?" → describes the Object
- Q3: "If I change this, what else changes?" → tests independence

Example applied:

Type: Student → Class is "Student"

Name="Sokha", GPA=3.8, Major="CS" → this is a Student Object

If Sokha's GPA changes → no other student object is affected → ✓ independent

Part 4 — Java as Illustration (10 min)

Reminder to students: We are not studying Java syntax right now.
We are seeing how the OOP concept looks when expressed in code.
Read the code as concept descriptions, not as programming instructions.

Reading a Class as a Concept Description

```
// This is the CLASS – the template
// Read it as: "Every Student will have a name, age, and major.
//           Every Student can introduce themselves and study."

public class Student {

    // ATTRIBUTES – every Student will have these
    String name;    // filled in later by each specific object
    int    age;     // filled in later by each specific object
    String major;   // filled in later by each specific object

    // ACTIONS – every Student can do these
    void introduce() {
        // uses THIS specific object's own name and age
    }
}
```

```
void study() {      System.out.println(name + " is studying " + major);  }  
}
```

Creating Objects — Filling in the Template

```
public class Main {
    public static void main(String[] args) {

        // BUILD OBJECT 1 - give the template real data
        Student sokha = new Student();
        sokha.name = "Sokha";
        sokha.age = 20;
        sokha.major = "Computer Science";

        // BUILD OBJECT 2 - same template, completely different data
        Student dara = new Student();
        dara.name = "Dara";
        dara.age = 22;
        dara.major = "Engineering";

        // Each object performs actions using its OWN data
        sokha.introduce();    // → Hi, I am Sokha, age 20
        dara.introduce();    // → Hi, I am Dara, age 22

        // PROVE INDEPENDENCE
        sokha.name = "Sokha Chenda";    // change only Sokha's name
        sokha.introduce();    // → Hi, I am Sokha Chenda, age 20
        dara.introduce();    // → Hi, I am Dara, age 22 ← unchanged
    }
}
```

How to Read One Line of Code as Three Concepts

```
Student sokha = new Student();
```

↑ ↑ ↑

| | └ "Build a real Student object in memory"

| | (construct the actual house)

| └ "Call this specific object 'sokha'"

| (the label on the address)

└ "This object's type is Student"

(it follows the Student blueprint)

The University Model for This Course

All code examples this semester use one domain: a university.

WHAT WE BUILD – WEEK BY WEEK

Week 2 (now): Student class, Course class
Week 3: Deepen attributes, methods, object state
Week 4: Protect data with Encapsulation
Week 5: Construct objects cleanly
Week 6–7: Teacher class, Person parent class
Week 9–11: Connect everything with Polymorphism
Week 12: Model how Student, Teacher, Course relate

By Week 15: a complete university OOP application

Test Checkpoint

Concept checks — no Java syntax required:

- ☐ Can explain what a Class is in one sentence without using the word “Java”
- ☐ Can explain what an Object is in one sentence without using the word “Java”
- ☐ Can give a new real-world example (not Student, not Car) of a class and 2 objects from it
- ☐ Can identify which is the Class and which is the Object in any given description
- ☐ Can explain WHY changing one object does not affect another object of the same class
- ☐ Can read `Student sokha = new Student()` and explain each part conceptually

Common Misconceptions

Misconception	Clarification
“Class and Object are the same thing”	Class = template, no data. Object = real instance, actual data. One recipe, many meals cooked from it.

"Two objects with same data are the same object"	No. Sokha and Dara can both have GPA 3.8 — they are still two different independent objects.
"The class automatically creates objects"	No. You must explicitly build each object from the class — the blueprint does not build houses by itself.
"You can only create one object per class"	No. One class → unlimited objects. One cookie cutter → unlimited cookies.

Extension Challenges

● Basic — Class or Object?

For each item, state whether it is a **Class** or an **Object**. Give one sentence explaining why.

1. "Smartphone" (the general concept)
2. "Dara's iPhone 15 Pro, 45% battery, 256GB storage"
3. "A bank account owned by Sokha with balance \$350 and account number KH-0042"
4. "BankAccount" (the general concept)
5. "The specific coffee order: Large iced Americano, no sugar, extra ice, order #47"
6. "DrinkOrder" (the general concept)

● Intermediate — Design a Class

You are building an app for a bookshop in Phnom Penh (no coding needed — plain English or a diagram):

- Design a `Book` class: list 5 attributes and 3 actions
- Create 3 real book objects with actual values for all attributes
- Sokha buys one book (sets `isSold = true`) — which other books are affected? Why?
- What happens to the `Book` class when Sokha buys the book? Why?

● Advanced — One Class, Many Contexts

The same concept "Person" is used in many different systems. Consider three different systems.

For each system, what attributes and actions would a `Person` class have?

- A **university management system**
- A **hospital patient management system**
- A **bank customer management system**

Write a short paragraph: should these all be one shared `Person` class, or three separate classes? What are the trade-offs? What OOP concept from later weeks might help? (Hint: Week 6.)

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. A **CLASS** is a template – it defines structure only.
It says: "every object of this type will have **THESE** attributes and can do **THESE** actions."
A class stores **NO** actual data.
 2. An **OBJECT** is a real, specific instance built from a class.
It holds **ACTUAL** data values.
It can perform all actions defined in its class.
 3. Objects are **INDEPENDENT**.
Every object owns its own copy of every attribute.
Changing one object has **NO** effect on any other object.
The class is never affected by changes to any object.
 4. Object **STATE** = the current values of all attributes.
State changes over time as the object performs actions.
 5. One class → as many objects as you need.
Cookie cutter → unlimited cookies.
Student class → every student in the university system.
-

Homework (No Java required)

Design a `Smartphone` class:

1. List **6 attributes** every smartphone has
2. List **4 actions** every smartphone can perform
3. Create **3 smartphone objects** (Sokha's, Dara's, Chan's) with real values for all 6 attributes
4. Write 3–5 sentences: "If Sokha's battery drops to 5%, does that affect Dara's or Chan's phone? Why? Which OOP principle explains this?"

No code this week. Pure concept understanding first.

Bring your design to Week 3 — we will use it to explore how object state changes over time.

Week 2 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com